

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
по дисциплине
«Информатика и программирование»

Студент		
гр. БИН-25-3	_____	Т. Г. Пономаренко
Ассистент		
преподавателя	_____	М.В. Водяницкий

Задание

Выполнить задания и оформить отчет по стандартам ВВГУ.

Задание 1. Написать функцию, которая конвертирует время из одной величины в другую. На вход подаются:

- число (величина времени)
- исходная единица измерения
- единица измерения, в которую нужно перевести

Функция должна вернуть конвертированное значение.

Пример:

Введите время для конвертации: 4 h m

Результат: 240m

Задание 2. Пользователь делает вклад в банке в размере a рублей сроком на n лет. Процент по вкладу зависит от суммы и срока.

Зависимость от суммы:

- каждые 10 000 рублей увеличивают ставку на 0.3%
- суммарное увеличение не может превышать 5%
- минимальный вклад — 30 000 рублей

Зависимость от срока:

- первые 3 года — 3%
- от 4 до 6 лет — 5%
- более 6 лет — 2%

Необходимо написать функцию, которая рассчитывает **прибыль пользователя без учета первоначально вложенной суммы**. Используется сложный процент.

Пример:

Ввод: 30000 3

Прибыль: 3648.67 руб.

Задание 3. Написать функцию для вывода всех простых чисел в заданном диапазоне. Нужно учитывать некорректные данные (например, начало больше конца или диапазон без простых чисел).

На вход подаются два числа: начало и конец диапазона (включительно). На выходе — список всех простых чисел или сообщение об ошибке.

Пример:

Начало диапазона: 1

Конец диапазона: 10

2 3 5 7

Пример:

Начало диапазона: 0

Конец диапазона: 1

Error!

Задание 4. Реализовать функцию сложения двух матриц. При сложении двух матриц получается новая матрица того же размера, где каждый элемент — это сумма элементов с тем же индексом из двух исходных матриц.

Ограничения:

- складывать можно только матрицы одинакового размера
- размер матрицы должен быть строго больше 2 (например, 3×3, 4×4 и т.д.)
- при нарушении условий нужно вывести сообщение об ошибке

На вход подаются:

- 1) размер матрицы n (для квадратной матрицы $n \times n$)
- 2) элементы первой матрицы (по строкам, через пробел)
- 3) элементы второй матрицы (в таком же формате)

Результат — новая матрица (в том же формате), либо сообщение об ошибке.

Задание 5. Написать функцию, которая определяет, является ли строка палиндромом. Палиндром — это строка, которая читается одинаково слева направо и справа налево (без учета пробелов, регистра и знаков препинания).

На вход подается строка. На выходе:

- Да, если это палиндром
- Нет, если это не палиндром

Пример:

А роза упала на лапу Азора

Да

Пример:

Алфавитный порядок

Нет

Содержание

1	Выполнение работы	3
1.1	Задание 1	3
1.2	Задание 2	3
1.3	Задание 3	4
1.4	Задание 4	5
1.5	Задание 5	6

1 Выполнение работы

1.1 Задание 1

1. Объявление функции - Создаём функцию `convert_time` с тремя параметрами 2. Проверка входной единицы "часы Если исходная единица равна "h" 3. Расчёт секунд из часов - Умножаем число на 3600 (секунд в одном часе) 4. Проверка входной единицы "минуты Если исходная единица равна "m", 5. Расчёт секунд из минут - Умножаем число на 60 (секунд в одной минуте) 6. Случай для секунд - Если исходная единица равна "s", оставляем без изменений 7. Другой случай - Если единица не распознана, считаем что это секунды 8. Проверка выходной единицы "часы Если целевая единица равна "h", 9. Расчёт часов из секунд - Делим общее количество секунд на 3600 10. Проверка выходной единицы "минуты Если целевая единица равна "m", 11. Расчёт минут из секунд - Делим общее количество секунд на 60 12. Случай для секунд на выходе - Если целевая единица равна "s", оставляем без изменений 13. Другой случай - Если единица не распознана, оставляем секунды 14. Вывод результата - Преобразуем число в строку и добавляем целевую единицу измерения. На рисунке 1 представлен код программы.

```

1 def convert_time(n, source_unit, target_unit):
2     # Переводим в секунды
3     if source_unit == "h":
4         total_seconds = n * 3600
5     elif source_unit == "m":
6         total_seconds = n * 60
7     elif source_unit == "s":
8         total_seconds = n
9     else:
10        total_seconds = n
11
12    if target_unit == "h":
13        result = total_seconds / 3600
14    elif target_unit == "m":
15        result = total_seconds / 60
16    elif target_unit == "s":
17        result = total_seconds
18    else:
19        result = total_seconds
20
21    print(str(result) + target_unit)

```

Рисунок 1 – Листинг программы для задания 1

1.2 Задание 2

1. Объявление функции - Создаём функцию 'vklad' с двумя параметрами (сумма и срок) 2. Проверка минимальной суммы - Проверяем, что сумма больше или равна 30000 3. Вывод ошибки при малой сумме - Если сумма меньше 30000, выводим "Error!" и завершаем 4. Расчёт бонусной ставки - За каждые 10000 добавляем 0.35. Ограничение бонусной ставки - Максимальный бонус не может превышать 56. Определение базовой

ставки по сроку - Для срока до 3 лет: 37. Расчёт общей процентной ставки - Складываем базовую и бонусную ставки 8. Инициализация переменной для накопления - Начинаем с исходной суммы 9. Инициализация счётчика лет - Устанавливаем начальное значение 0 10. Цикл начисления процентов - Повторяем для каждого года вклада 11. Начисление процентов за год - Увеличиваем сумму на процент от текущей суммы 12. Увеличение счётчика лет - Переходим к следующему году 13. Расчёт прибыли - Вычитаем из итоговой суммы первоначальный вклад 14. Вывод результата с округлением - Округляем прибыль до двух знаков после запятой На рисунке 2 представлен код программы.

```

1 def vklad(s, years):
2     if s < 30000:
3         print("Error!")
4         return
5
6     bonus = s // 10000 * 0.3
7     if bonus > 5:
8         bonus = 5
9
10    if years <= 3:
11        base = 3
12    elif years <= 6:
13        base = 5
14    else:
15        base = 2
16
17    total_rate = base + bonus
18
19    money = s
20    year = 0
21    while year < years:
22        money = money + money * total_rate / 100
23        year = year + 1
24
25    profit = money - s
26    print(round(profit, 2))
27 print(vklad(70000,5))

```

Рисунок 2 – Листинг программы для задания 2

1.3 Задание 3

1. Объявление функции - Создаём функцию 'primes' с двумя параметрами (начало и конец диапазона)
2. Создание пустого списка - Инициализируем список для хранения простых чисел
3. Цикл по числам диапазона - Перебираем числа от начала до конца включительно, начиная минимум с 2
4. Проверка числа на простоту - Рассматриваем только числа больше 1
5. Поиск делителей - Проверяем делители числа от 2 до квадратного корня из числа
6. Проверка делимости - Если число делится на какой-либо делитель без остатка
7. Выход из внутреннего цикла - Прерываем проверку, так как число составное
8. Добавление простого числа - Если цикл завершился без прерывания, добавляем число в список
9. Возврат результата - Возвращаем список простых чисел или "Error!" если список пуст
10. Ввод начала диапазона - Запрашиваем у пользователя начало диапазона
11. Ввод

конца диапазона - Запрашиваем у пользователя конец диапазона 12. Вывод результата - Выводим все простые числа через пробел На рисунке 3 представлен код программы.

```

1 def primes(a,b):
2     sp = []
3     for num in range(max(2, a), b + 1):
4         if num > 1:
5             for i in range (2, int(num**0.5) + 1):
6                 if num % i == 0:
7                     break
8             else:
9                 sp.append(num)
10
11     return sp if sp else "Error!"
12
13 a = int(input("Начало диапазона:"))
14 b = int(input("Конец диапазона:"))
15 print(*primes(a, b))
16
17 primes(a,b)

```

Рисунок 3 – Листинг программы для задания 3

1.4 Задание 4

1. Объявление функции - Создаём функцию без параметров 2. Начало блока обработки ошибок - Помещаем код в конструкцию 'try' для отлова исключений 3. Ввод размера матрицы - Запрашиваем у пользователя размер квадратной матрицы 4. Проверка минимального размера - Проверяем, что размер больше 2 5. Вывод ошибки при малом размере - Если размер меньше или равен 2, выводим "Error!"и завершаем 6. Ввод первой матрицы - Запрашиваем элементы первой матрицы построчно 7. Преобразование строки в список чисел - Разбиваем строку на числа и создаём список 8. Создание матрицы 1 - Формируем список списков (матрицу) из введённых строк 9. Ввод второй матрицы - Запрашиваем элементы второй матрицы построчно 10. Создание матрицы 2 - Формируем вторую матрицу аналогичным образом 11. Вывод заголовка результата - Печатаем "Результат:"перед выводом суммы матриц 12. Цикл по строкам матрицы - Перебираем строки от 0 до n-1 13. Сложение элементов матриц - Для каждого столбца складываем элементы из обеих матриц 14. Формирование строки результата - Преобразуем суммы в строки и соединяем пробелами 15. Вывод строки результата - Печатаем каждую строку результирующей матрицы 16. Обработка ошибок ввода - Если возникает любая ошибка, переходим в блок 'except' 17. Вывод сообщения об ошибке - При любой ошибке выводим "Error!"18. Вызов функции - Запускаем выполнение функции На рисунке 4 представлен код решения.

```

1
2 def add_m():
3     try:
4         n = int(input("Введите размер матрицы: "))
5
6         if n <= 2:
7             print("Error!")
8             return
9
10        print("Введите первую матрицу:")
11        m1 = [list(map(int, input().split())) for _ in range
12              (n)]
13        print("Введите вторую матрицу:")
14        m2 = [list(map(int, input().split())) for _ in range
15              (n)]
16        print("Результат:")
17        for i in range(n):
18            print(' '.join(str(m1[i][j] + m2[i][j]) for j in
19                            range(n)))
20    except:
21        print("Error!")
22 add_m()

```

Рисунок 4 – Листинг программы для задания 4

1.5 Задание 5

1. Объявление функции - Создаём функцию без параметров 2. Ввод строки - Запрашиваем строку и удаляем пробелы по краям 3. Создание пустой строки - Инициализируем строку для очищенного текста 4. Очистка строки - Убираем все символы кроме букв и цифр, переводим в нижний регистр 5. Создание перевёрнутой строки - Получаем строку в обратном порядке 6. Сравнение строк - Проверяем, равна ли строка своему перевёрнутому варианту 7. Вывод результата - Если строки равны: "Да иначе: "Нет" 8. Запуск функции - Вызываем функцию для выполнения На рисунке 5 представлен код программы.

```

1 def check_palindrome():
2     s = input().strip()
3     cleaned = ""
4
5     for c in s:
6         if c.isalnum():
7             cleaned += c.lower()
8
9     reversed_cleaned = cleaned[::-1]
10
11    if cleaned == reversed_cleaned:
12        print("Да")
13    else:
14        print("Нет")
15
16 check_palindrome()

```

Рисунок 5 – Листинг программы для задания 5